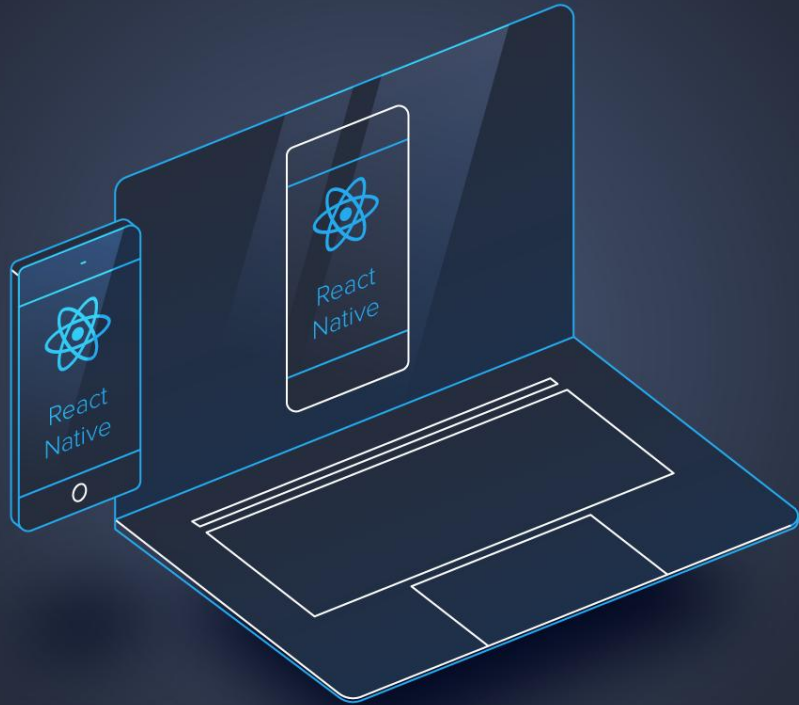




# ASZTALI ÉS MOBIL ALKALMAZÁSOK FEJLESZTÉSE ÉS TESZTELÉSE TANTÁRGY – MOBIL ALKALMAZÁSOK RÉSZ

13. A osztály

Tanárok: Dr. Koncz Adrienn és  
Simon Martin

2025.09.18.

---

# REACT NATIVE – STYLESHEET KOMPONENS

- StyleSheet a React Native beépített modulja, amelyet a komponensek stílusának meghatározására használnak.
- Ez tulajdonképpen egy JavaScript objektum alapú stílusdefiníció, ami hasonló a CSS-hez, de teljesen JavaScript-ben van megvalósítva.
- Hivatalos import:

```
import { StyleSheet } from 'react-native';
```

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Helló React Native!</Text>
      <Text style={styles.paragraph}>Ez egy bekezdés.</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1, // kitölti a teljes képernyőt
    backgroundColor: '#f0f0f0',
    alignItems: 'center', // vízszintesen középre
    justifyContent: 'center', // függőlegesen középre
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: 'darkblue',
    marginBottom: 10,
  },
  paragraph: {
    fontSize: 16,
    color: 'black',
  },
});
```

---

# SZINTAXIS ÉS HASZNÁLAT

- A StyleSheet egy JavaScript objektumot hoz létre, amely kulcs–érték párokat tartalmaz.
- A kulcs a stílus neve, az érték pedig egy másik objektum, ami a CSS-hez hasonló property–value párokat tartalmaz.

---

# HASZNÁLAT FORMÁI

```
const styles = StyleSheet.create({
  box: {
    width: 100,
    height: 100,
    backgroundColor: 'red',
  }
});
```

## 1. StyleSheet.create()

- Létrehozza a stílusokat validált módon, ami a hibás property-t kiszűri.

---

# HASZNÁLAT FORMÁI

## 2. Inline stílus

- Nem kell külön StyleSheet.
- Kis projektekhez vagy gyors prototípushoz jó.

```
<Text style={{ color: 'blue', fontSize: 20 }}>Kék szöveg</Text>
```

---

# HASZNÁLAT FORMÁI

## 3. Több stílus kombinálása

- Többel lehet több stílust összeadni.
- Ha ugyanaz a property többször szerepel, a későbbi felülírja az előzőt.

```
<Text style={[styles.title, { color: 'green' }]}>Zöld cím</Text>
```

---

# KÜLÖNBSÉGEK A STYLESHEET ÉS A CSS KÖZÖTT

- **camelCase** → minden property JS stílusban van, pl. backgroundColor (nem background-color).
  - **Nincs minden CSS tulajdonság** → csak azok, amiket a React Native támogat.
  - **Nincs osztálynév** → a stílusnevek JS kulcsok, pl. styles.container.
  - **Egység nélküli számok** → px helyett sima számot írunk (fontSize: 20 → 20px-nek felel meg).
  - **Flexbox az alap** → az elrendezés mindig flexboxra épül (default display: flex minden <View>-ban).
-

---

# GYAKRAN HASZNÁLT CSS TULAJDONSÁGOK, AMIKET NEM TÁMOGAT A REACT NATIVE

- **float** – nincs lebegtetés, helyette flexboxot kell használni
  - **z-index** – alap szinten van (zIndex), de nem olyan összetett, mint weben
  - **position: sticky** – nem elérhető, csak absolute, relative
  - **display: block/inline/inline-block** – nem létezik. Csak display: none és alapértelmezett flex
  - **filter (pl. blur, brightness)** – nincs általánosan, de van blurRadius képeknél
  - **background-image** – nincs, helyette ImageBackground komponens
  - **Pseudo-szelektorok (:hover, :focus, :after, :before)** – teljesen hiányoznak
-

---

# MÉRTÉKEGYSÉGEK REACT NATIVE-BAN

Ami támogatott:

- **Egyszerű számok** → **px-ként értendők**
  - Példa: `fontSize: 20` → 20 px a képernyő skálázás szerint.
  - **Soha nem kell "20px" formátumot írni**, mert string nem érvényes.
- **Százalékok** – bizonyos property-knél működnek (`width`, `height`, `margin`, `padding`, `top`, `left` stb.).  
Példa: `width: '80%'`.



---

# MÉRTÉKEGYSÉGEK REACT NATIVE-BAN

## Ami NEM támogatott:

- **em, rem** – nincs betűméret-alapú relatív mértékegység.
- **vh, vw** – nincs viewport-relative egység.  
Ehelyett a Dimensions API-t kell használni:
- **pt** – iOS-en a „pont” natív mértékegység, de a React Native ezt automatikusan kezeli → nem lehet explicit pt-t írni, csak számot.
- **cm, mm, in, % of font-size** – nem léteznek.

```
import { Dimensions } from 'react-native';
const { width, height } = Dimensions.get('window');

const styles = StyleSheet.create({
  box: {
    width: width * 0.8,    // 80vw helyett
    height: height * 0.5, // 50vh helyett
  }
});
```

---

# PÉLDÁK MÉRTÉKEGYSÉGEKRE

```
const styles = StyleSheet.create({
  title: {
    fontSize: 20,           // = 20 px (automatikusan DP-re skálázva)
    margin: '10%',          // százalék támogatott
  },
  box: {
    width: '50%',           // fél szélesség
    height: 100,            // fix 100 px
  }
});
```

---

---

# PÉLDÁK MÉRTÉKEGYSÉGEKRE

- Ha szeretnél em vagy rem viselkedést, azt manuálisan kell megvalósítani:
  - Definiálj egy `baseFontSize` változót, és szorozd fel vele:

```
const baseFontSize = 16;

const styles = StyleSheet.create({
  text: {
    fontSize: 1.5 * baseFontSize, // ~1.5em
  }
});
```

---

# MI AZ A DP?

- A DP (Density-independent Pixel) a mobilfejlesztésben használt alap mértékegység, ami nem ugyanaz, mint a sima képpont (px).
  - Ez egy absztrakt mértékegység, amit a mobil operációs rendszerek (Android, iOS) használnak, hogy a UI elemek mindenféle kijelzőméreten és pixelsűrűségen (ppi/dpi) arányosan ugyanakkorának látszódjanak.
  - 1 dp körülbelül 1 pixelnek felel meg egy 160 dpi sűrűségű kijelzőn (ez az alapértelmezett "baseline density").
-

---

# MIÉRT NEM SIMA PX?

- Ha egyszerű px-t használnánk, akkor:
  - Egy 300 px széles gomb egy alacsony felbontású telefonon majdnem képernyőszélességű lenne.
  - Egy nagy felbontású (pl. 4K) telefonon viszont aprócska gomb lenne ugyanaz a 300 px.
- A DP ezt a problémát oldja meg azzal, hogy a rendszer automatikusan skálázza a számokat a készülék DPI-je (dots per inch) alapján.

---

# HOGYAN MŰKÖDIK A SKÁLÁZÁS?

Androidon például a képlet:

$$px = dp \times (dpi/160)$$

- Ha egy telefon **160 dpi** (régebbi, alacsony sűrűség)  $\rightarrow 1\text{ dp} = 1\text{ px}$ .
  - Ha egy telefon **320 dpi** (tipikus mai mobil)  $\rightarrow 1\text{ dp} = 2\text{ px}$ .
  - Ha egy telefon **480 dpi** (QHD kijelző)  $\rightarrow 1\text{ dp} = 3\text{ px}$ .
  
  - iOS-ben nem "dp"-nek hívják, hanem pt (points).
  - A logika ugyanaz: egy "point" több pixelnek felelhet meg retina kijelzőn.
    - iPhone 8 (326 ppi):  $1\text{ pt} = 2\text{ px}$
    - iPhone 13 Pro (460 ppi):  $1\text{ pt} = 3\text{ px}$
  - React Native ezeket egységesen kezeli, így fejlesztőként nem kell külön számolnod, csak számot adsz meg  $\rightarrow$  és az automatikusan DP/pt szerint fog skálázódni.
-

---

# HOGYAN MŰKÖDIK A SKÁLÁZÁS?

```
const styles = StyleSheet.create({  
  box: {  
    width: 100,    // 100 dp → minden készüléken kb. ugyanakkora doboz  
    height: 50,    // 50 dp  
    backgroundColor: 'blue',  
  },  
});
```

Ez nem 100 pixel lesz fixen, hanem a rendszer "átváltja" annyi natív pixelre, amennyire szükség van, hogy vizuálisan arányos maradjon.

---

---

# FLEXBOX - [HTTPS://REACTNATIVE.DEV/DOCS/FLEXBOX](https://reactnative.dev/docs/flexbox)

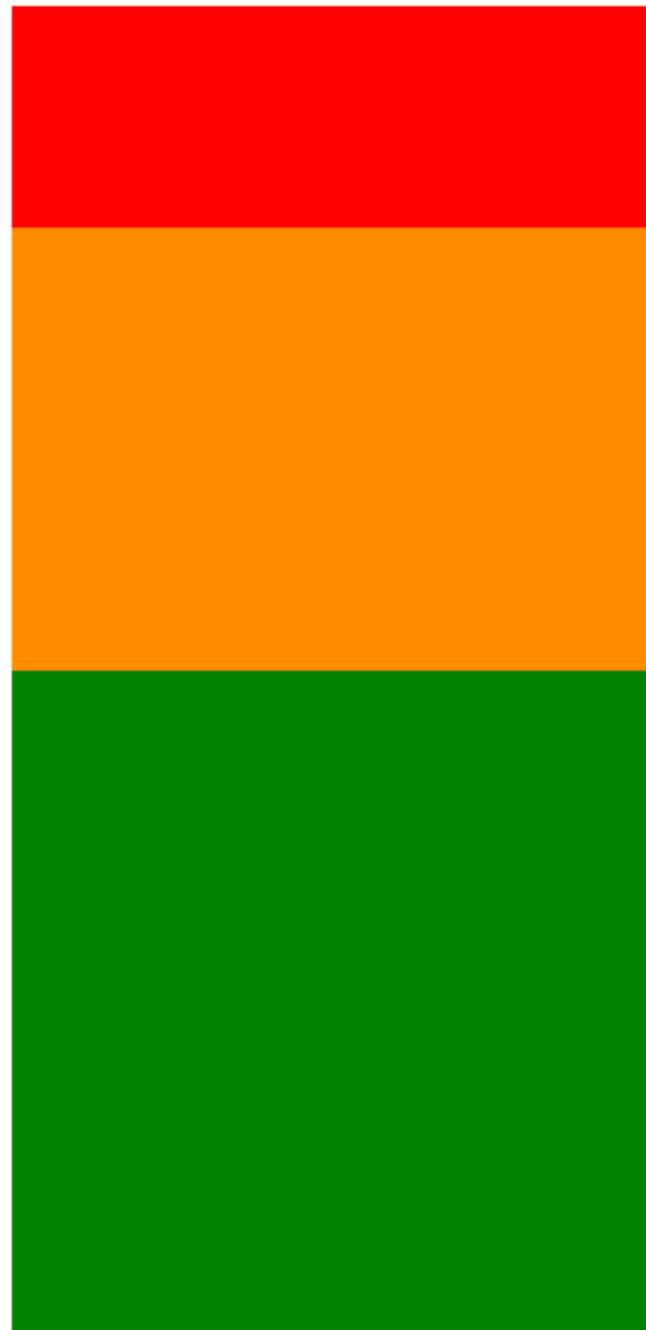
- Egy komponens a gyermekelemeinek elrendezését a **Flexbox algoritmus** segítségével tudja meghatározni.
- A Flexbox úgy lett tervezve, hogy **következetes elrendezést biztosítson különböző képernyőméreteken**.
- Általában a **flexDirection**, **alignItems** és **justifyContent** kombinációját használjuk a megfelelő elrendezés eléréséhez.
- **Fontos!** A Flexbox ugyanúgy működik a React Native-ban, mint a webes CSS-ben, néhány kivétellel:
- Az alapértelmezések eltérőek:
  - **flexDirection** alapértelmezetten **column**, nem pedig **row**.
  - **alignContent** alapértelmezetten **flex-start**, nem pedig **stretch**.
  - **flexShrink** alapértelmezetten **0**, nem pedig **1**.
  - A flex paraméter csak **egyetlen számot** támogat.

```
import React from 'react';
import {StyleSheet, View} from 'react-native';

const Flex = () => {
  return (
    <View
      style={[
        styles.container,
        {
          // Try setting `flexDirection` to `"row"`.
          flexDirection: 'column',
        },
      ]>
      <View style={{flex: 1, backgroundColor: 'red'}} />
      <View style={{flex: 2, backgroundColor: 'darkorange'}} />
      <View style={{flex: 3, backgroundColor: 'green'}} />
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
});

export default Flex;
```



---

# FLEX DIRECTION

- A **flexDirection** szabályozza, hogy egy elem gyermekei milyen irányban legyenek elrendezve. Ezt a tengelyt nevezzük **főtengelynek (main axis)**. A **keresztirányú tengely (cross axis)** a főtengelyre merőleges irány, vagyis az, amely mentén a sortörések (wrapping) történnek.
  - **column** (*alapértelmezett érték*): A gyermekek **fentről lefelé** rendeződnek. Ha a sortörés engedélyezve van, akkor a következő sor a konténer tetején, az első elem jobb oldalán kezdődik.
  - **row**: A gyermekek **balról jobbra** rendeződnek. Ha a sortörés engedélyezve van, akkor a következő sor az első elem alatt, a konténer bal oldalán kezdődik.
  - **column-reverse**: A gyermekek **lentől felfelé** rendeződnek. Ha a sortörés engedélyezve van, akkor a következő sor a konténer alján, az első elem jobb oldalán kezdődik.
  - **row-reverse**: A gyermekek **jobbról balra** rendeződnek. Ha a sortörés engedélyezve van, akkor a következő sor az első elem alatt, a konténer jobb oldalán kezdődik.
-

---

# LAYOUT DIRECTION

- A **layout direction** (elrendezési irány) meghatározza, hogy a hierarchiában lévő gyermekelemek és szövegek **milyen irányban legyenek elrendezve**. Az elrendezési irány azt is befolyásolja, hogy a **start** és **end** szélek mire vonatkoznak.
  - Alapértelmezés szerint a React Native **LTR (left-to-right, balról jobbra)** elrendezést használ.
  - Ebben a módban:
    - a start a **bal oldalt**,
    - az end a **jobb oldalt** jelenti.
  - **Értékek:**
    - **LTR (alapértelmezett érték):** A szöveg és a gyermekelemek **balról jobbra** rendeződnek. Az elem margin és padding értékei, amelyek a start oldalra vonatkoznak, a **bal oldalra** kerülnek.
    - **RTL:** A szöveg és a gyermekelemek **jobbról balra** rendeződnek. Az elem margin és padding értékei, amelyek a start oldalra vonatkoznak, a **jobb oldalra** kerülnek.
-

---

# JUSTIFYCONTENT ÉS ALIGNITEMS

React Native-ban a **justifyContent** és az **alignItems** egyaránt a Flexbox rendszer részét képezik, de különböző módon befolyásolják az elrendezést:

- **justifyContent:** A fő tengely (main axis) mentén igazítja a gyermekelemeket.
- **alignItems:** A keresztirányú tengely (cross axis) mentén igazítja a gyermekelemeket.



---

# JUSTIFYCONTENT - (FŐ TENGELY SZERINTI IGAZÍTÁS)

- A fő tengely irányát az `flexDirection` határozza meg:
    - Ha **`flexDirection`**: "**`column`**" (alapértelmezett), akkor a fő tengely függőleges (felülről lefelé).
    - Ha **`flexDirection`**: "**`row`**", akkor a fő tengely vízszintes (balról jobbra).
  - Lehetséges értékek:
    - **`flex-start`** (alapértelmezett) – Az elemek a fő tengely elejére kerülnek.
    - **`center`** – Az elemek középre kerülnek a fő tengelyen.
    - **`flex-end`** – Az elemek a fő tengely végére kerülnek.
    - **`space-between`** – Az első és az utolsó elem a szélekre kerül, a köztes elemek között egyenlő térköz lesz.
    - **`space-around`** – Minden elem körül egyenlő térköz lesz.
    - **`space-evenly`** – Minden elem között egyforma térköz lesz, beleértve az első és az utolsó elem közötti teret is.
-

---

# ALIGNITEMS - (KERESZTIRÁNYÚ TENGELY SZERINTI IGAZÍTÁS)

- A keresztirányú tengely mindig merőleges a flexDirection által meghatározott fő tengelyre.
- Lehetséges értékek:
  - **flex-start** – Az elemek a keresztirányú tengely elejére igazodnak.
  - **center** – Az elemek középre kerülnek a keresztirányú tengelyen.
  - **flex-end** – Az elemek a keresztirányú tengely végére igazodnak.
  - **stretch** (alapértelmezett) – Az elemek kitöltik a rendelkezésre álló helyet a keresztirányú tengely mentén (ha nincs megadva szélesség/magasság).
  - **baseline** – Az elemek szövegének alapvonala mentén igazodnak.

Ahhoz, hogy a stretch beállításnak legyen hatása, a gyermekelemeknek nem szabad fix mérettel rendelkezniük a keresztirányú tengely mentén. A weboldalon lévő példában az alignItems: 'stretch' nem fejt ki semmilyen hatást, amíg a gyermekeknél szereplő width: 50 el nem távolítjuk.

---

---

# ALIGN CONTENT

Az **alignContent** meghatározza a sorok elosztását a **keresztirányú tengely** mentén. Ez a beállítás **csak akkor fejt ki a hatását**, ha az elemek több sorba törnek a flexWrap használatával.

**Értékek:**

- **flex-start (alapértelmezett érték)** A törött sorok a konténer keresztirányú tengelyének elejéhez igazodnak.
  - **flex-end:** A törött sorok a konténer keresztirányú tengelyének végéhez igazodnak.
  - **stretch** *(alapértelmezett érték a Yoga használatakor a weben)* A törött sorok kinyúlnak, hogy kitöltsék a konténer keresztirányú tengelyének teljes magasságát.
  - **center:** A törött sorok a konténer keresztirányú tengelyének közepére igazodnak.
  - **space-between:** A törött sorok egyenletesen helyezkednek el a konténer keresztirányú tengelyén, a fennmaradó helyet elosztva a sorok között.
  - **space-around:** A törött sorok egyenletesen helyezkednek el a konténer keresztirányú tengelyén, a fennmaradó helyet elosztva a sorok körül. A konténer két szélén a térköz fele akkora, mint a sorok közötti térköz.
  - **space-evenly:** A törött sorok egyenletesen helyezkednek el a konténer keresztirányú tengelyén, a fennmaradó helyet elosztva a sorok körül. Minden térköz pontosan **ugyanakkora méretű**.
-

---

# ROW GAP, COLUMN GAP ÉS GAP

- **rowGap**: beállítja a **sorok közötti távolságot** egy elemben.
  - **columnGap**: beállítja az **oszlopok közötti távolságot** egy elemben.
  - **gap**: beállítja a **sorok és oszlopok közötti távolságot**. Ez egy rövidítés a rowGap és a columnGap számára.
  - A flexWrap és az alignContent használatával a gap segítségével **egységes térközt** adhatsz az elemek közé.
-

---

# WIDTH ÉS HEIGHT

A **width** tulajdonság egy elem **szélességét**, a **height** pedig az elem **magasságát** határozza meg a tartalmi területén.

Mind a width, mind a height a következő értékeket veheti fel:

- **auto (alapértelmezett érték):** A React Native kiszámítja az elem szélességét/magasságát a tartalom alapján – legyen az más gyermekelem, szöveg vagy kép.
  - **pixels:** Fix szélességet/magasságot ad meg abszolút pixelekből. Az elem tényleges mérete azonban más stílusbeállításoktól is függhet, ezért előfordulhat, hogy nem pontosan ez lesz a végső méret.
  - **percentage (%):** A szélességet vagy magasságot a szülő elem szélességének/magasságának **százalékában** adja meg.
-

---

# REACT NATIVE – IMAGE KOMPONENS

- Egy React komponens különböző típusú képek megjelenítéséhez, beleértve:
  - hálózati képeket,
  - statikus erőforrásokat,
  - ideiglenes helyi képeket,
  - valamint a helyi lemezről (például a képgalériából) származó képeket.
- Az alábbi példa bemutatja, hogyan lehet képet betölteni és megjeleníteni:
  - **helyi tárhelyről,**
  - **URI-vel**
  - és még a 'data:' URI-sémával megadott **adatból** is.
- **Megjegyzés:** Hálózati és adat URI-ból betöltött képek esetén **kézzel kell megadnod a kép méreteit!**

---

# REACT NATIVE – IMAGE KOMPONENS

**Helyi fájl beszúrása:**

```
<Image source={require('./assets/logo.png')} style={{ width: 120, height: 120 }} />
```

**URI-vel való beszúrás:**

```
<Image source={{ uri: 'https://example.com/pic.png' }} style={{ width: 300, height: 180 }} />
```

**data: uri séma:**

```
<Image source={{ uri: 'data:image/png;base64,iVBORw0K...' }} style={{ width: 100, height: 100 }} />
```

Ez a verzió csak kis képeknél ajánlott (memória/JS méret miatt).

---

---

# KÉPEK FORMÁZÁSA

## Méretezés & arány

- width, height: fix méret.
- aspectRatio: szélesség/magasság arány. Szuper reszponzív trükk:

```
<Image source={{ uri }} style={{ width: '100%', aspectRatio: 16/9 }} />
```

```

import React from "react";
import { View, Image, StyleSheet, Dimensions } from "react-native";

const windowHeight = Dimensions.get("window").width;

const ExampleComponent = () => {
  return (
    <View style={styles.container}>
      <Image
        source={require('./940178.jpg')}
        style={styles.image}
        resizeMode="contain"
      />
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
  },
  image: {
    width: windowHeight * 0.7, // Adjust the percentage as needed
    aspectRatio: 1, // Maintain the aspect ratio of the image
  },
});

export default ExampleComponent;

```

**aspectRatio tulajdonság:** Az aspectRatio tulajdonsággal megadhatja a kép képarányát. Ez segít a kép arányos méretezésében, függetlenül a képernyőmérettől.

Ez tulajdonképpen a kép szélességének és magasságának a hányadosa. (pl. 16/9)

---

# KÉPEK FORMÁZÁSA

## Kivágás és illesztés – resizeMode

- **cover:** kitölti a keretet, levághat széleket.
- **contain:** teljes kép látszik, lehet üres sáv.
- **stretch:** széthúzza, arány torzulhat.
- **center:** középre teszi, nem skáláz automatikusan.
- **repeat:** ismétlődés (iOS). → szerintem Android-on is működik

```
<Image
  source={{ uri }}
  style={{ width: 200, height: 120 }}
  resizeMode="cover" // 'cover' | 'contain' | 'stretch' | 'center' | 'repeat' (repeat: iOS)
/>
```

---

---

# RESPONZÍV MEGJELENÍTÉS

A "reszponzív" azt jelenti, hogy egy weboldal vagy alkalmazás képes alkalmazkodni a különböző képernyőméretekhez és eszközökhöz, így optimális felhasználói élményt nyújt minden platformon.



---

# DIMENSIONS MODUL

```
<Image
  source={require("../assets/gif/SearchingDrivers.gif")}
  style={{
    height: windowHeight * 0.25,
    width: windowWidth * 0.3,
    // Add any additional styling if needed
  }}
/>
```

- A kódban a **windowHeight** és a **windowWidth** az eszköz kijelzőjének méreteit jelenti.
  - Ha ezeket a kívánt százalékkal megszorozod, a kép alkalmazkodóvá válik, és méretei a View szélességéhez igazodnak.
  - Ha például  $\text{windowWidth} * 0.3$ -mal számolsz, akkor a kép a View szélességének 30%-át fogja elfoglalni, míg  $\text{windowHeight} * 0.25$  beállítja a kép magasságát a kijelző ablakának 25%-ára. Ezeket az arányokat a kívánt elrendezésnek megfelelően módosíthatod.
  - Ezzel a változtatással a kép a nézet szélességéhez arányosan méreteződik, és megőrzi a reszponzivitását különböző képernyőméretek esetén.
  - <https://medium.com/@Sarmilasivaraja/creating-responsive-images-in-react-native-1194bf036e98>
-

---

# MIÉRT SZOKÁS IMAGE-ET VIEW-BA ÁGYAZNI?

- Gyakori, hogy Image komponenst View-ba ágyazzák, de ez mindig attól függ, mit akarunk elérni.
- Alapból az Image egy önálló komponens, tehát nem kötelező köré View, de sok helyzetben praktikus vagy éppen szükséges.
- Példák, amikor szükséges az Image-t View-ba ágyazni:

## 1. Layout és elrendezés miatt

- A View flexbox-szal jobban kontrollálható, mint egy Image.
- Ha több elem (pl. szöveg, gomb) kerül a kép mellé vagy köré, sokkal könnyebb egy szülő View-ban elrendezni őket. Pl kép + név + leírás egy kártyán.

```
<View style={styles.card}>
  <Image source={require('./avatar.png')} style={styles.avatar} />
  <Text style={styles.name}>Anna</Text>
</View>
```

---

---

## 2. Háttér és stílus trükkök

- Ha a képet keretben, árnyékkal, padding-gal akarsz megjeleníteni, akkor azt általában a View-ra rakod.
- Maga az Image nem kezeli jól pl. az árnyékot Androidon (elevation), de a View igen.

```
<View style={styles.imageWrapper}>  
  <Image source={uri} style={styles.image} />  
</View>
```

---

### 3. Overlay (ráhelyezett tartalom)

- Ha szöveget, ikont, gombot szeretnél a képre rátenni, akkor ImageBackground vagy View szükséges.
- **Az Image NEM képes gyerekeket megjeleníteni!**
- Példa View-val:

```
<View style={{ position: 'relative' }}>
  <Image source={require('./bg.jpg')} style={{ width: '100%', height: 200 }} />
  <Text style={{ position: 'absolute', bottom: 10, left: 10, color: 'white' }}>
    Szöveg a képen
  </Text>
</View>
```

---

## 4. Interakciók kezelése

- Ha a képre kattintani lehet (pl. profilkép → profiloldal), akkor a képet általában `TouchableOpacity` vagy `Pressable`-be burkolják, ami szintén `View`-ra épül.

```
<TouchableOpacity onPress={onPress}>  
  <Image source={require('./pic.jpg')} style={{ width: 100, height: 100 }} />  
</TouchableOpacity>
```

6:52

Shiba Inu



A siba inu japán kutyafajta, a hat eredeti spitz fajta közül a legkisebb. Kistermetű, mozgékony kutya, jól alkalmazkodik a hegyvidéki terephez, ezért eredetileg vadászkutyaként tenyésztették. Hasonlít az akitára de kisebb. Egyike azon kevés ősi kutyafajtáknak, amelyek mind a mai napig fennmaradtak. Az inu japánul kutyát jelent, de a siba előtag eredete még nem tisztázott. A siba szó jelentése „bozót”, pontosabban olyan fa vagy bokor, mely levelei ősszel vörös színűvé válnak. A legtöbb siba inu szőre vörös színű, árnyalata megegyezik a bokrokéval, ezért sokan úgy gondolják, innen ered az elnevezésük. Mások úgy vélik, magára a bozótosra utal, mivel ezeket a kutyákat ott használták leginkább vadászatra. Tovább bonyolítja a helyzetet, hogy egy régi, naganói dialektusban a siba szó „kicsit” jelent, elképzelhető tehát, hogy a természetük után nevezték el őket. Éppen ezért előfordul a „kis, bozóti kutya” fordítás is.

6:54



A siba inu japán kutyafajta, a hat eredeti spitz fajta közül a legkisebb. Kistermetű, mozgékony kutya, jól alkalmazkodik a hegyvidéki terephez, ezért eredetileg vadászkutyaként tenyésztették. Hasonlít az akitára de kisebb. Egyike azon kevés ősi kutyafajtáknak, amelyek mind a mai napig fennmaradtak. Az inu japánul kutyát jelent, de a siba előtag eredete még nem tisztázott. A siba szó jelentése „bozót”, pontosabban olyan fa vagy bokor, mely levelei ősszel vörös színűvé válnak. A legtöbb siba inu szőre vörös színű, árnyalata megegyezik a bokrokéval, ezért sokan úgy gondolják, innen ered az elnevezésük. Mások úgy vélik, magára a bozótosra utal, mivel ezeket a kutyákat ott használták leginkább vadászatra. Tovább bonyolítja a helyzetet, hogy egy régi, naganói dialektusban a siba szó „kicsit” jelent, elképzelhető tehát, hogy a természetük után nevezték el őket. Éppen ezért előfordul a „kis, bozóti kutya” fordítás is.



# GYAKORLÓ FELADATOK

Image gyakorlás

6:52

Shiba Inu



A siba inu japán kutyafajta, a hat eredeti spitz fajta közül a legkisebb. Kistermetű, mozgékony kutya, jól alkalmazkodik a hegyvidéki terephez, ezért eredetileg vadászkutyaként tenyésztették. Hasonlít az akitára de kisebb. Egyike azon kevés ősi kutyafajtáknak, amelyek mind a mai napig fennmaradtak. Az inu japánul kutyát jelent, de a siba előtag eredete még nem tisztázott. A siba szó jelentése „bozót”, pontosabban olyan fa vagy bokor, mely levelei ősszel vörös színűvé válnak. A legtöbb siba inu szőre vörös színű, árnyalata megegyezik a bokrokéval, ezért sokan úgy gondolják, innen ered az elnevezésük. Mások úgy vélik, magára a bozótosra utal, mivel ezeket a kutyákat ott használták leginkább vadászatra. Tovább bonyolítja a helyzetet, hogy egy régi, naganói dialektusban a siba szó „kicsit” jelent, elképzelhető tehát, hogy a termetük után nevezték el őket. Éppen ezért előfordul a „kis, bozóti kutya” fordítás is.

6:52



A siba inu japán kutyafajta, a hat eredeti spitz fajta közül a legkisebb. Kistermetű, mozgékony kutya, jól alkalmazkodik a hegyvidéki terephez, ezért eredetileg vadászkutyaként tenyésztették. Hasonlít az akitára de kisebb. Egyike azon kevés ősi kutyafajtáknak, amelyek mind a mai napig fennmaradtak. Az inu japánul kutyát jelent, de a siba előtag eredete még nem tisztázott. A siba szó jelentése „bozót”, pontosabban olyan fa vagy bokor, mely levelei ősszel vörös színűvé válnak. A legtöbb siba inu szőre vörös színű, árnyalata megegyezik a bokrokéval, ezért sokan úgy gondolják, innen ered az elnevezésük. Mások úgy vélik, magára a bozótosra utal, mivel ezeket a kutyákat ott használták leginkább vadászatra. Tovább bonyolítja a helyzetet, hogy egy régi, naganói dialektusban a siba szó „kicsit” jelent, elképzelhető tehát, hogy a termetük után nevezték el őket. Éppen ezért előfordul a „kis, bozóti kutya” fordítás is.



# GYAKORLÓ FELADATOK

Image gyakorlás

# GYAKORLÓ FELADATOK

## Image gyakorlás

### Hogyan gondozzuk a sünöket a kertben?



A sünök apró, de lenyűgöző lények, amelyek egészen jól érzik magukat a kertjeinkben. Az apró, tüskés sünök főként éjszakai állatok, amelyek napközben elbújnak. Éjszaka előbújnak, hogy táplálékot keressenek, és táplálékuk főként rovarokból, csigákból és gerinctelen állatokból áll.

Bátoríthatjuk ezeket az imádnivaló teremtményeket, hogy látogassanak el a kertünkbe. Egy jól kialakított kerthelyiség barátságos lesz ezeknek a kis emlősöknek, és büszkéek lehetünk arra, hogy sün van az udvarunkon.



### Építsünk sünházat - barátságos kert a sünnek

hogy látogassanak el a kertünkbe. Egy jól kialakított kerthelyiség barátságos lesz ezeknek a kis emlősöknek, és büszkéek lehetünk arra, hogy sün van az udvarunkon.



### Építsünk sünházat - barátságos kert a sünnek

Magunk is készíthetünk sünházat, nem egy bonyolult projekt. Készítsünk egy fadobozt alj nélkül, és tegyünk rá megfelelő lejtésű tetőt. Helyezzük a dobozt egy csendes helyre, és töltsük meg szénával. Ez a legegyszerűbb projekt, amit megcsinálhatunk. A piacon kaphatók kész sünházak is. Válasszunk természetes anyagokból - fából vagy fonottból - készült modelleket. A ház nemcsak természetes menedéket nyújt majd a sünnek, hanem a sün téli odúja is lesz.



# GYAKORLÓ FELADATOK

StyleSheet gyakorlás



---

# ÖNÁLLÓ MUNKA / HÁZI FELADAT

- Készíts egy egyszerű screent, ahol bemutatsz a hobbid.
  - Kötelező elemek:
    - A fő cím nagyobb betűméretben, középre igazítva, színes legyen
    - Legyen 3 kép → az egyik legyen mindenképpen uri-s, egy másik pedig saját fájlból szúrd be, a 3. opcionális. A képek méretére figyelj oda.
    - Legyen több (2-3) szöveges rész, ahol bemutatsz a hobbid. A szöveges részek legyenek sorkizártak, és jól láthatóak (legyen min. 14-es méret).
    - Legyen a screen görgethető
    - Ha vannak alcímeid, azokat balra igazítsd, de az is legyen más színű, nagyobb méretű.
    - Állítsál be háttérszínt is, amint jól olvasható a szöveg.
    - Használj tetszőleges elrendezési megoldást is (flexbox, igazítás).
    - **Az elkészült feladatokat a Moodle-be fel kell tölteni!**
-